16.02.2022.

Curs 1

Nota
$\frac{2}{3}$ ex.
$\frac{1}{3}$ lab.

Suport de curs.

- Cormen → Introduction to algorithms

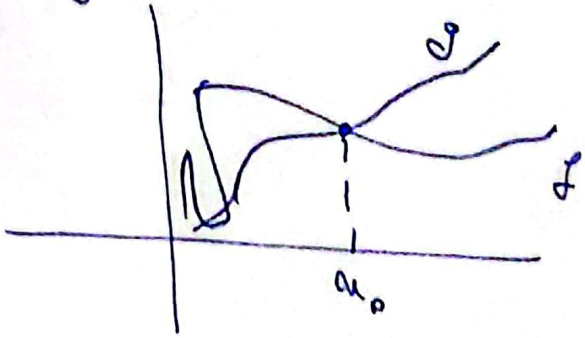
Notatii asimptotice	$o, O, \Theta, \Omega, \omega$

Când proiectăm algoritmi avem în vedere
→ eficienta: spatiu de memorie
timp de rulare
→ corectitudine

Timpul de rulare = nr. de operații

o → o mic
O → o mare
Ω → omega
ω → omega mic
Θ → theta

$O(g)$ = (informal) mulțimea funcțiilor care cresc mai încet sau la fel de încet ca funcția g.



Scanned with CamScanner

Definitie:

Spunem că o funcție $f \in O(g)$ dacă $\exists n_0, c > 0$ a.î. $\forall n \geq n_0$ avem $f(n) \leq c \cdot g(n)$

Exemple:

- $2n \in O(n^2)$ $\quad n_0 = 2$, $c = 1$
  $f(n) = 2n$, $g(n) = n^2$
  $\forall n \geq 2 : 2n \leq 1 \cdot n^2$

- $200n^2 \in O(n^2)$ Adevărat
  $n_0 = 1$ $\quad \forall n \geq 1 : 200n^2 \leq c \cdot n^2$
  $c = 200$

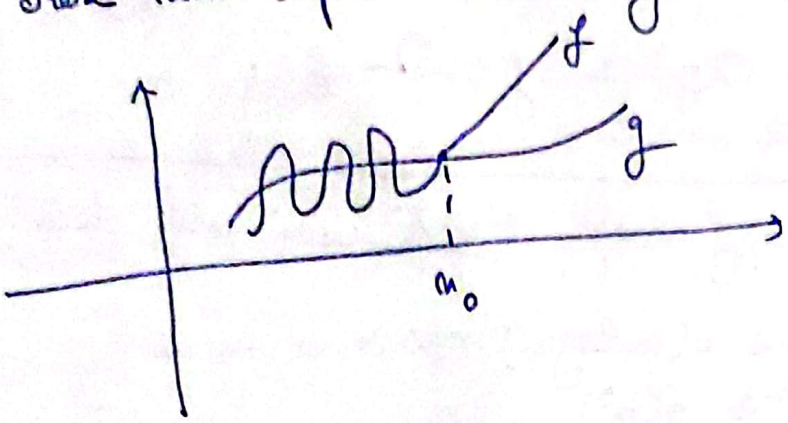
- $n^2 \notin O(n)$
  Pp. rod. abs. $n^2 \in O(n) \Rightarrow \exists n_0, c > 0$ a.î. $n^2 < c \cdot n$
  $\forall n \geq n_0$. $\quad \Downarrow$ $n < c$ ↯
  pt-că c = constantă.

- $2^{n+1} \in O(2^n)$ ✓
  $c = 2$
  $n_0 = 1$

- $2^{2n} \in O(2^n)$ NU
  Temă acasă.

Scanned with CamScanner

Ω = (informal) reprezintă mulțimea funcțiilor care cresc mai repede sau la fel de repede ca g.



Exemple: $n^2 \in \Omega(n \log n)$ | $2^n \in \Omega(\log n)$

$\frac{n}{2} \in \Omega(n)$ |

Definiție:

Spunem că $f \in \Omega(g)$ dacă $\exists\, n_0, c > 0$ a.î.

$\forall n > n_0$ avem $f(n) \geq c \cdot g(n)$.

$f \in \Theta(g)$ = (informal) : f crește la fel de repede ca g.

Definiție:

Spunem că $f \in \Theta(g)$ dacă $\exists\, n_0, c_1, c_2 > 0$ a.î.

$\forall n > n_0$ avem : $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

Exemple: $200 n^2 \notin \Theta(n)$ $\quad n \notin \Theta(n \log n)$

$200 n^2 \in \Theta(n^2)$

$\frac{n^2}{10} \in \Theta(n^2)$

Scanned with CamScanner

Teoremă: (finitivat)

$f \in \Theta(g) \Leftrightarrow f \in O(g)$ și $f \in \Omega(g)$.

$f \in o(g)$ = (informal) f crește strict mai încet decât g.

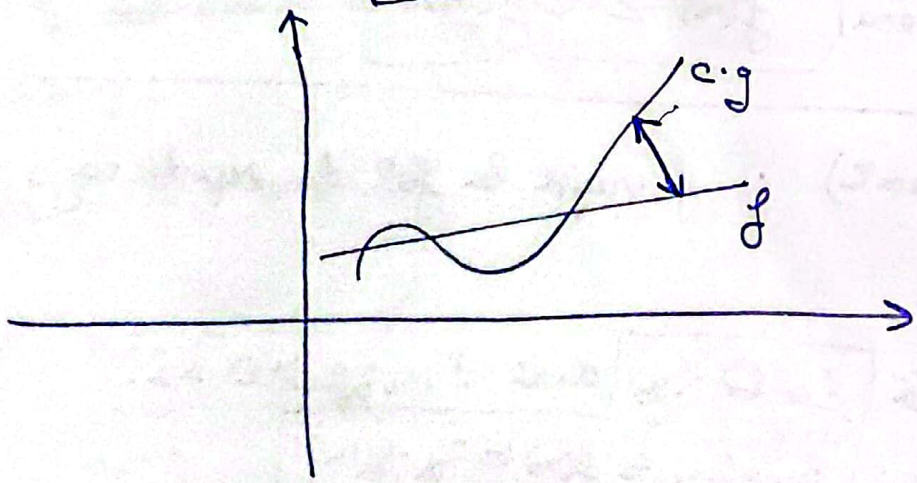
Exemple: $n \in o(n \log n)$

$n^2 \in o(n^3)$

$n \notin o(n)$ $\quad \frac{n}{2} \notin o(n)$

$n \notin o(100n)$

Definiție: $f \in o(g)$ dacă $\forall c > 0 : \exists n_0 > 0$ a.î. $\forall n \geq n_0$ avem $f(n) < c \cdot g(n)$



$f \in \omega(g)$ = (informal) f crește strict mai repede decât g.

Exemple: $n^3 \in \omega(n^2)$

$2^n \in \omega(n)$

Definiție: $f \in \omega(g)$ dacă $\forall c > 0 : \exists n_0 > 0$ a.î. $\forall n \geq n_0$ avem $f(n) > c \cdot g(n)$

Scanned with CamScanner

Exercitiu:

$\Rightarrow n \in o(n^2)$

Fie $c > 0$ fixat.

Vrem să găsim un n_0 a.î. $\forall n > n_0$ să avem $n < c \cdot n^2$.

* n_0 depinde de c; de exemplu: putem alege $n_0 = \frac{1}{c} \Rightarrow$

$\Rightarrow \forall n > \frac{1}{c}$ avem $n < c \cdot n^2$.

$\Rightarrow n! \in o(n^n)$ (exc.) * aplicăm log inamurile părți $\Rightarrow$

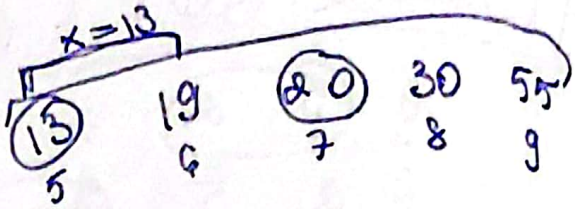
$\log n! \in \Theta(n \log n)$ (exc.)

~~Seminar 1~~

Scanned with CamScanner

(I) Căutare secvențială

(II) Căutare binară:



(0+9)/2 → 4 10 == 13 ?
13 > 10

(5+9)/2 → 7 20 == 13 ?
13 < 20

(5+6)/2 → 5 13 == 13 ? DA.

23.02.2022

Curs 2

Analiza timpului de rulare a algoritmilor recursivi.

Exemple: metode de sortare

- Quick
- Merge
- Bubble
- Heap
- Radix
- Insertion
- Count

→ SORT → n log n

→ n^2

→ sunt bazați pe comparații între chei

Scanned with CamScanner

Insertion sort recursive

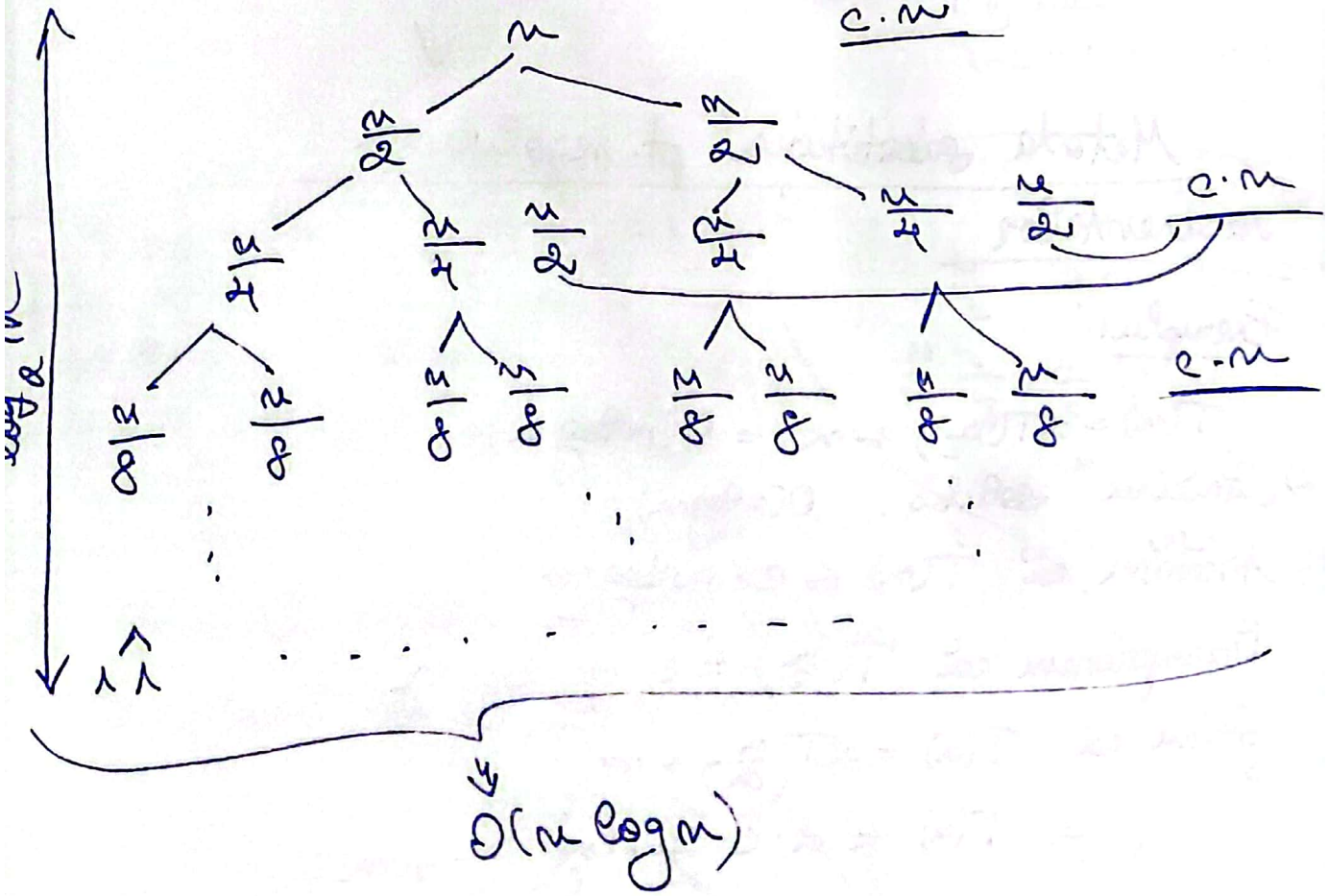
$$T(n) = T(n-1) + O(n) = O(n^2)$$

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$$

$$= 1 \quad \frac{n}{3} + T\left(\frac{n}{3^2}\right)$$

Merge sort

$$T(n) = 2T\left(\frac{n}{2}\right) + \overset{c \cdot n}{O(n)} = \Theta(n \log n)$$



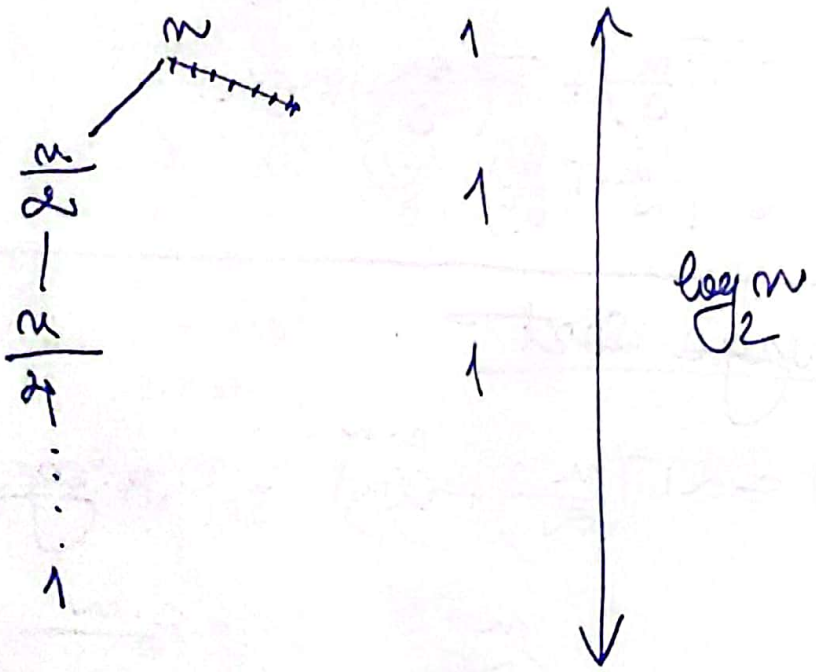
$$\Downarrow$$

$$O(n \log n)$$

Scanned with CamScanner

Căutarea binară:

$$T(n) = T\left(\frac{n}{2}\right) + 1 \Rightarrow O(\log n)$$



Metoda substituției pt. rezolvarea recurențelor

Exemplu:

$$T(n) = 2T\left(\frac{n}{2}\right) + n = O(n \log n)$$

1) „Ghicim" soluția. $O(n \log n)$

2) Arătăm că $T(n) \leq c \cdot n \log_2 n$

Presupunem că $T\left(\frac{n}{2}\right) \leq c \cdot \frac{n}{2} \cdot \log_2 \frac{n}{2}$

Știm că $T(n) = 2T\left(\frac{n}{2}\right) + n$

$$\Rightarrow T(n) \leq 2 \cdot c \cdot \frac{n}{2} \cdot \log_2 \frac{n}{2} + n$$

$$\Rightarrow T(n) \leq c \cdot n \log_2 \frac{n}{2} + n$$

$$\Rightarrow T(n) \leq c \cdot n \cdot (\log_2 n - 1) + n$$

$$\Rightarrow T(n) \leq c \cdot n \cdot \log_2 n - c \cdot n + n$$

Scanned with CamScanner

$\Rightarrow T(n) \leq cn\log_2 n + n(1-c)$

Avem $T(n) \leq cn\log_2 n$ dacă $1-c \leq 0$

Ad. pt. $c \geq 1$

$T(n) = T\left(\frac{n}{2}\right) + 1 = O(\log_2 n)$

Vrem să arătăm că $T(n) \leq c \cdot \log_2 n$

Presupunem că $T\left(\frac{n}{2}\right) \leq c \cdot \log_2 \frac{n}{2}$

~~Și vrem arătăm că~~ $T(n) \leq c \cdot \log_2 \frac{n}{2} + 1$

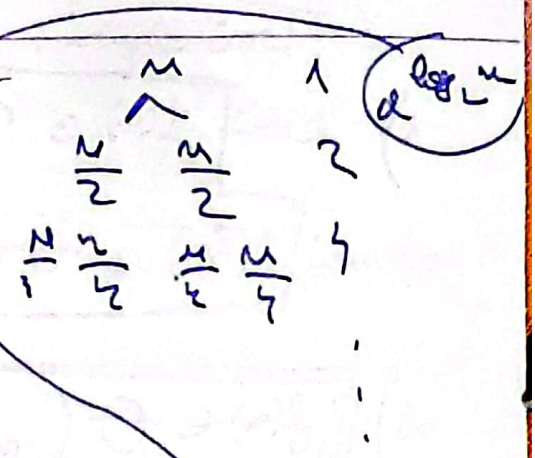
$\Rightarrow T(n) \leq c\log_2 n - c + 1$

$\Rightarrow T(n) \leq c\log_2 n + (1-c)$

↳ dacă

$1-c \leq 0 \Rightarrow c \geq 1$

$T(n) = 2T\left(\frac{n}{2}\right) + 1 = O(n)$



$1 + 2 + 4 + \ldots = O(n)$

$\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots = O(\log_2 n)$

Vrem să arătăm că $T(n) \leq c \cdot n$

Presupunem că $T\left(\frac{n}{2}\right) \leq \frac{cn}{2}$

$\Rightarrow T(n) \leq 2 \cdot c \cdot \frac{n}{2} + 1$

$\Rightarrow T(n) \leq cn + 1$

Vrem să arătăm că $T(n) \leq cn - d$

Presupunem că $T\left(\frac{n}{2}\right) \leq c\frac{n}{2} - d$ $O(n-1)$

Scanned with CamScanner

$\Rightarrow T(n) \leq c \cdot \frac{n}{2} - d) + 1$

$\Rightarrow T(n) \leq cn - 2d + 1$

$\Leftrightarrow T(n) \leq cn + \underbrace{(1-d)}_{\leq 0 \Rightarrow d \geq 1} - d$

$\Rightarrow T(n) \leq cn - d + (1-d)$

Teorema Master

Dacă avem recurența de forma:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Trei cazuri:

1) Dacă $f(n) \in O(n^{\log_b a - \varepsilon})$ pt. un $\varepsilon > 0$

atunci $T(n) \in \Theta(n^{\log_b a})$

2) $f(n) \in \Theta(n^{\log_b a})$ atunci

$$T(n) = \Theta(n^{\log_b a} \cdot \log_2 n)$$

3) $f(n) \in \Omega(n^{\log_b a + \varepsilon})$ pt. un $\varepsilon > 0$

și $a f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ pt. $c < 0$ atunci

$$T(n) \in \Theta(f(n))$$

Scanned with CamScanner

Exemple:

$T(n) = T\left(\frac{n}{2}\right) + 1 = \Theta(\log n)$

$f(n) = 1$

(caz 2)

$n^{\log_b a} = n^{\log_2 1} = 1$

$\Rightarrow f(n) \in \Theta(1)$

$T(n) = 9T\left(\frac{n}{3}\right) + n \qquad a = 9,\ b = 3$

$\Rightarrow f(n) = n$

$n^{\log_b a} = n^{\log_3 9} = n^2$

$(n^{\log_b a - \varepsilon})$

$\Rightarrow f(n) \in O(n^{2-\varepsilon})$ pt. $\varepsilon > 0$

(caz 1)

$\Rightarrow T(n) \in \Theta(n^2)$

$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$

$O(n \log n)$

nu pot aplica teorema Master

Scanned with CamScanner

02.03.2022.

Curs 3

Algoritmi probabilisti:
Analiza QuickSort:

Problema: | Secretary Problem | Strategia 1

Avem n candidați la un job pe care îi intervievăm pe rând. ~~Candidații~~ nu avem control asupra lor (nu îi putem sorta descrescător)

Dacă c_i (candidatul i) este mai bun decât toți candidații anteriori atunci îl/o angajăm și îi plătim un cost $\times$ (generat de concedierea celui anterior).

Câți candidați vom angaja?

* În cazul cel mai defavorabil vom angaja n candidați (când sunt sortați crescător).

Strategia 2: permutăm aleator candidații și aplicăm algoritmul anterior.

Care este nr. mediu de candidați angajați?

Scanned with CamScanner

Vom considera variabile aleatoare cu 2 valori 0 și 1.

Exemplu: dat cu banul

$$X = \begin{cases} 1, \text{ dacă pică „pajură”} \\ 0, \text{ altfel} \end{cases}$$

$P[X=1] = \frac{1}{2}$ - $P_r[X=0]$

Media unei variabile aleatoare: $\mathbb{E}[X] = 1 \cdot P[X=1] + 0 \cdot P[X=0]$ (expectation, entropie)

$= P_r[X=1]$

Ex.: Dăm de n ori cu banul. Probabilitatea ca moneda să pice pajură este p, altfel $1-p$.

Care este nr. mediu în care avem pajură?

$$X_i = \begin{cases} 1, \text{ dacă la a } i\text{-a aruncare avem pajură} \\ 0, \text{ altfel} \end{cases}$$

$X = X_1 + X_2 + \dots + X_n$

$\mathbb{E}[X] = \mathbb{E}[X_1 + X_2 + \dots + X_n] = \mathbb{E}[X_1] + \mathbb{E}[X_2] + \dots + \mathbb{E}[X_n]$ (și la produs doar dc. var. aleatoare sunt independente)

$= n \cdot \mathbb{E}[X_i] = n(1 \cdot \underbrace{P[X_i=1]}_{p} + 0 \cdot P[X_i=0]) = n \cdot p$

$$X_i = \begin{cases} 1, \text{ dacă angajăm candidatul } i \\ 0, \text{ altfel.} \end{cases}$$

Vrem să calculăm $X = X_1 + X_2 + \dots + X_n$.

$\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{n} X_i] = \sum_{i=1}^{n} \mathbb{E}[X_i]$

$= \sum_{i=1}^{n} P[X_i = 1] = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = O(\log n)$

Scanned with CamScanner

$X_{ij} = \begin{cases} 1, \text{ dacă studentul } i \text{ și studentul } j \text{ sunt} \\ \quad \text{născuți în aceeași zi.} \\ 0, \text{ altfel} \end{cases}$

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] \quad !$$

$$Pr[X_{ij} = 1] = \sum_{z=1}^{365} Pr[\text{st. } i \text{ să fie născut în ziua } z \text{ și st. } j \text{ să fie născut în ziua } z]$$

$= 365 \cdot Pr[\text{st. } i \text{ să fie născut în ziua } z] \cdot Pr[\text{st. } j \text{ să fie născut în ziua } z]$

Birthday paradox

$$= 365 \cdot \frac{1}{365} \cdot \frac{1}{365} = \frac{1}{365}$$

$$! \quad \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{1}{365} = \frac{n(n+1)}{2} \cdot \frac{1}{365}$$

Pt. $\frac{50 \cdot 51}{2} \cdot \frac{1}{365} > 1$

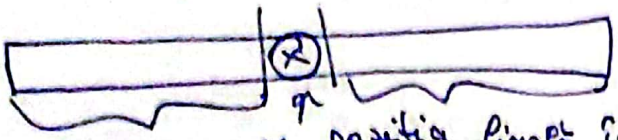
= în medie 4 perechi

Capitolul 5 & 7 (Cormen)

Scanned with CamScanner

QuickSort:

Ideea de bază:



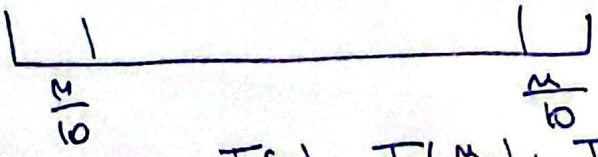
pe poziția finală în vectorul sortat

I. Alegem un pivot.
II. Apelăm funcția de partiție care așază pivotul pe poziția finală în vectorul sortat.
III. Apelăm recursiv QuickSort pe șirul din stânga, respectiv pe cel din dreapta pivotului.

Pivot determinist: $T(n) = T(n-1) + O(n) = O(n^2)$

↑ cazul defavorabil

În cazul favorabil: $T(n) = 2T\left(\frac{n}{2}\right) + O(n) = O(n \log n)$



Dacă am avea $T(n) = T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + O(n)$

$= O(n \log n)$

Similar $T(n) = T\left(\frac{n}{1000}\right) + T\left(\frac{999n}{1000}\right) + O(n)$

$= O(n \log n)$

Notăm elementele $z_1, z_2, \ldots z_n$, unde $z_1 \leq z_2 \leq z_3 \ldots \leq z_n$

$x_{ij} = \begin{cases} 1, \text{ dacă } z_i \text{ e comparat cu } z_j \\ 0, \text{ altfel.} \end{cases}$

Scanned with CamScanner

X = nr. de comparații efectuat de algoritm.

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} x_{ij}$$

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} x_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[x_{ij}]$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr[x_{ij} = 1]$$

unde $P[x_{ij} = 1]$ = Pr{ z_i sau z_j să fie ales ca pivot din intervalul $z_i, z_{i+1}, \ldots z_j$ }

$$= \frac{2}{j-i+1}$$

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \left(\sum_{k=1}^{n} \frac{2}{k+1} \right)$$

$k = j - i$

$k+i$

$O(\log n)$

$\cdot n$

$$= O(n \cdot \log n)$$

Scanned with CamScanner

for(i=n/2-1 ----)
// int.
// heapify (2k+1 sau 2k+2)

↓ $(n/2) \cdot \log n$

decapitare $n \cdot \log n$ +

$(3n/2) \log n \Rightarrow O(n \log n)$

09.03.2022.

Curs 4

1) Heap-uri (HeapSort)
2) Arbori binari de căutare:

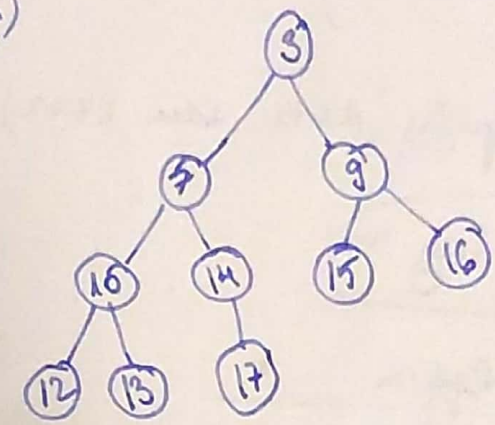
a) Ce operații suportă?
a1) Inserția unui element $O(\log n)$
a2) Extragerea minimului / maximului. $O(\log n)$
a3) Returnarea minimului / maximului $O(1)$

Un heap este un arbore binar plin (fiecare nod are 2 fii, cu excepția, posibil, a penultimului nivel)
cu prop. că valoarea din fiecare nod este mai mare decât toate valorile din subarbore.
(mică)

Min-Heap ⇒ fiecare nod va fi mai mic decât toate nodurile din subarbore.
Max-Heap ⇒ —//— mare —//—

Scanned with CamScanner

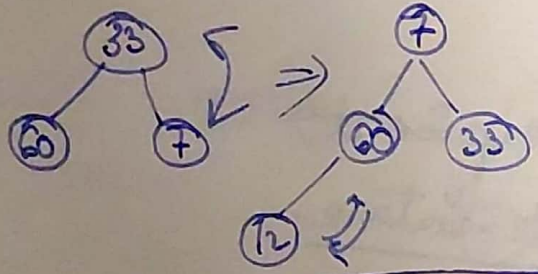
Exemplu: (min-heap)



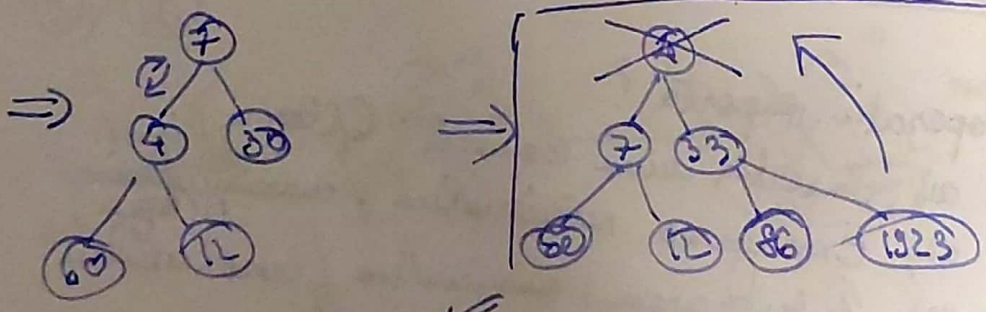
- O proprietate a heap-ului este că rădăcina va stoca minimul în Min-Heap.
maximul în Max-Heap.

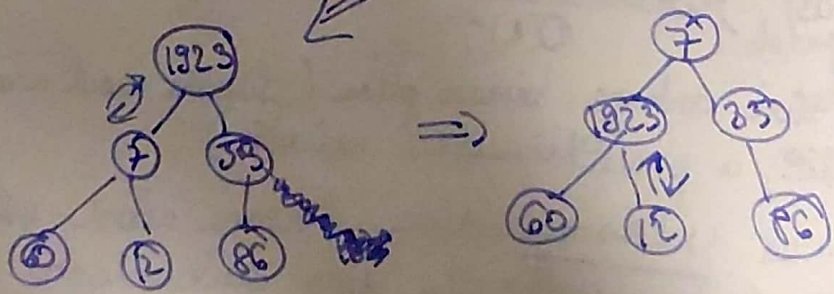
Exemplu: 33, 60, 7, 12, 4, 86, 1923 (min-heap)

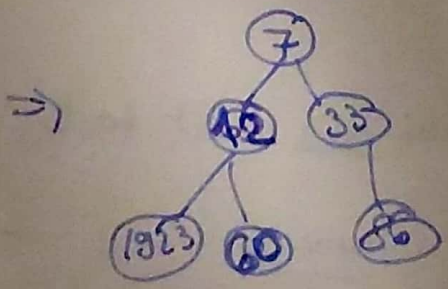
inserția unui element



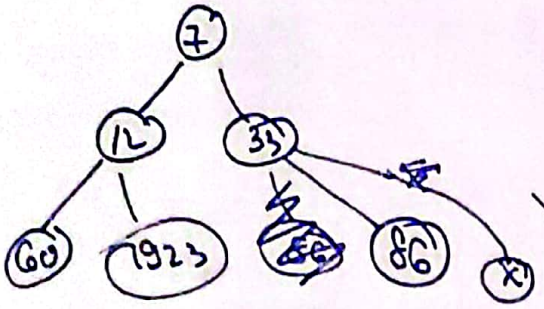
extragerea minimului







Scanned with CamScanner

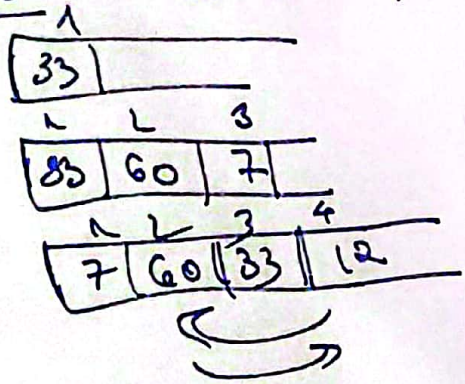


Sub formă de vector
2k+1, 2k+2

	1	2	3	4	5	6	7
V	7	12	33	60	1923	86	X

nod i → fiu stâng : 2i
→ fiu drept : 2i+1
→ tată [i/2]

Exemplu: 33 60 7 12 4 55

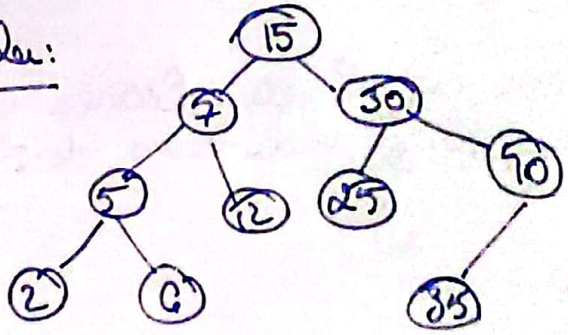


. . .

Arbori binari de căutare:

Def.: Un arbore binar de căutare este un arbore binar cu proprietatea că valoarea asociată fiecărui nod este mai mare decât toate valorile din subarborele stâng și mai mică decât toate valorile din subarborele drept.

Exemplu:

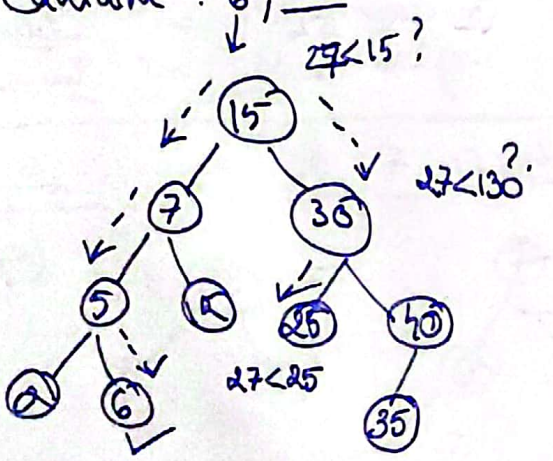


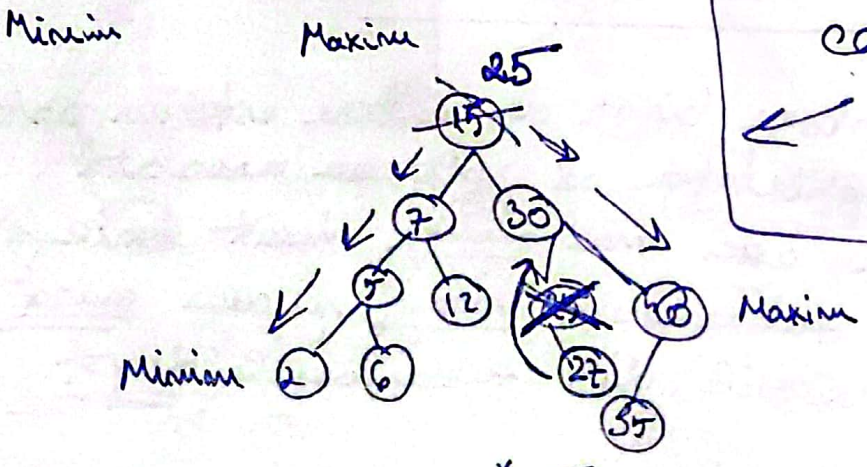
Scanned with CamScanner

Operații:

1) Inserție — $O(h)$
2) Căutare — $O(h)$
3) Min / Max — $O(h)$
4) Succesor / Predecesor — $O(h)$
5) Ștergere

Căutăm: 6, 27





cazul de la ștergere → ștergem 15

Succesorul unui nod este nodul cu valoarea cea mai mică, dar în același timp mai mare decât x.

$Succ(5) = 6$
$Succ(35) = 40$
$Succ(27) = 30$

Scanned with CamScanner

Predecesorul : — // — val. cea mai mare, mai mică decât x.

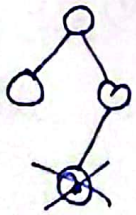
Pred(7) = 6
Pred(6) = 5
Pred(27) = 25

$$Succ(x) = \begin{cases} \text{min. din subarborele drept, dc. } x \text{ are fiu drept.} \\ \text{dc. } x \text{ nu are fiu drept., cel mai mic strămoș al cărui fiu stâng este tot strămoș al lui } > x. \end{cases}$$

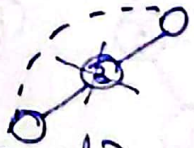
Pred(x)

Ștergerea:

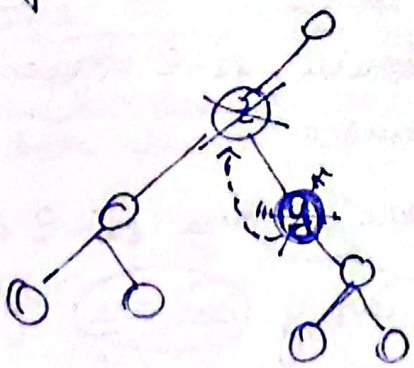
a) Frunză



b) Nodul nu are fiu drept.



c) Vrem să ștergem un nod x care are fiu drept, dar fiul drept nu are fiu stâng.



Scanned with CamScanner

d) Nodul t are fiu drept y care are fiu stâng.

16.03.2022.

Cursul 5

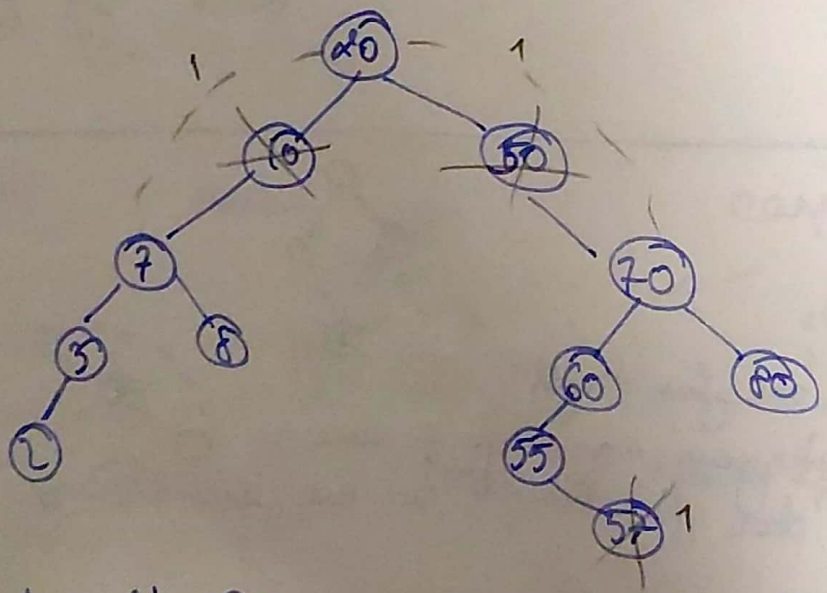
1) Recapitulare Lecția trecută: - stergere în arbori binari de căutare

2) Limite inferioare pt. sortare:

3) Count-Sort ; Radix-Sort

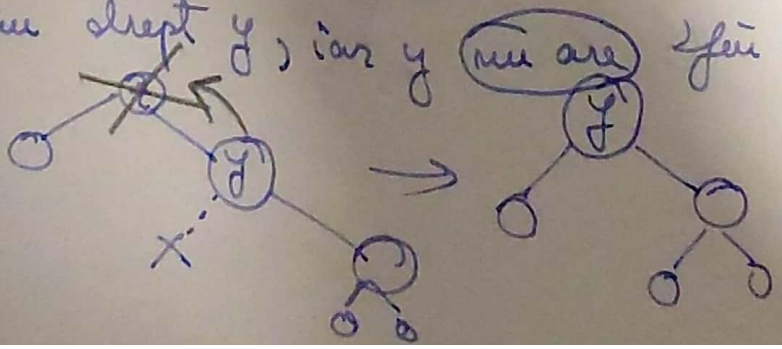
---

1)



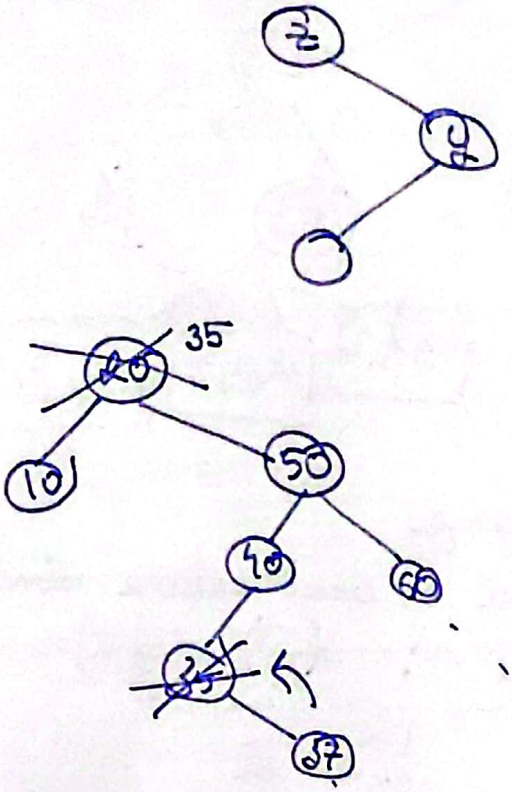
Caz 1: Nodul z pe care dorim să-l ștergem nu are fiu stâng sau fiu drept.

Caz 2: Nodul z pe care dorim să-l ștergem are un fiu drept y; iar y nu are fiu stâng.



Scanned with CamScanner

Caz. 3: Nodul z pe care dorim să-l ștergem are un fiu drept y, iar y are un fiu stâng.



Pași
1) Înlocuiesc z cu succ(z).
2) Șterg succ(z).

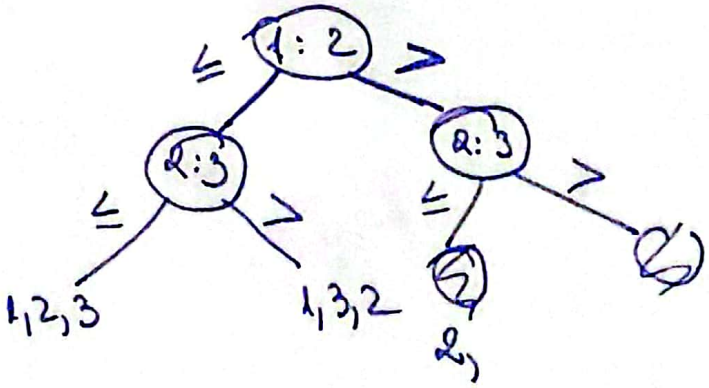
Corolar:

2) Teoremă: Orice algoritm de sortare bazat pe comparații într-o cheie efectuează $\Omega(n \log n)$ comparații.

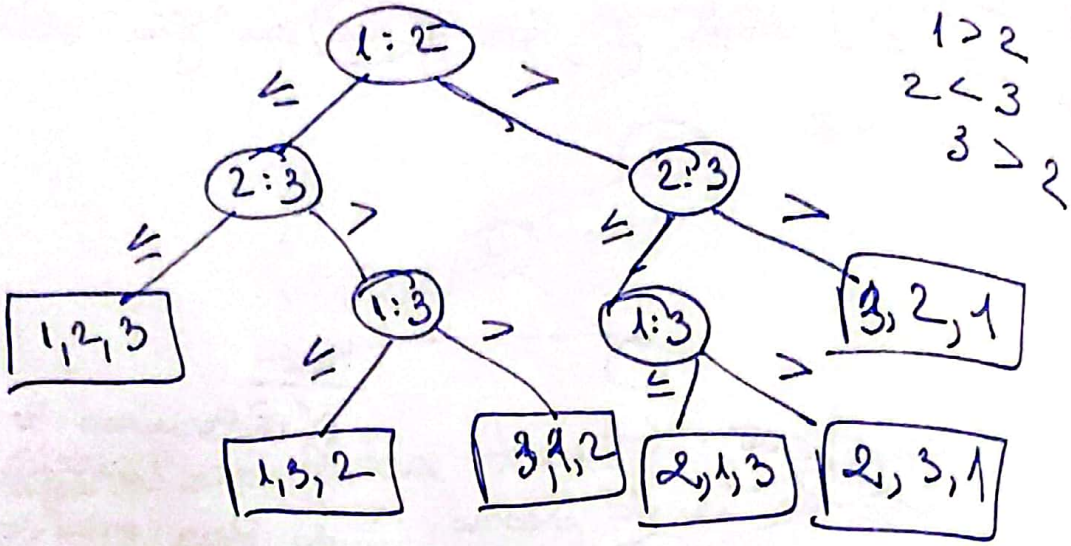
Demonstrație:

1) Un algoritm de sortare poate fi văzut modelat ca un arbore de decizie.

Ex de arbore de decizie pt. un vector cu 3 elemente: V[1], V[2], V[3]



Scanned with CamScanner



a) Arborele are $n!$ frunze.

3) Adâncimea minimă a unui arbore binar cu $n!$ frunze este $\log n! = \Omega(n \log n)$

$1 \cdot 2 \cdot \ldots \cdot n$
$n \cdot n \cdot \ldots \cdot n$

4) Timpul de rulare al unui algoritm ~~bazat~~ de sortare bazat pe chei este cel mai lung drum de la rădăcină la o frunză în arborele de decizie corespunzător.

$n - l$

Temă

$\log_2 n! = \Theta(n \log n)$

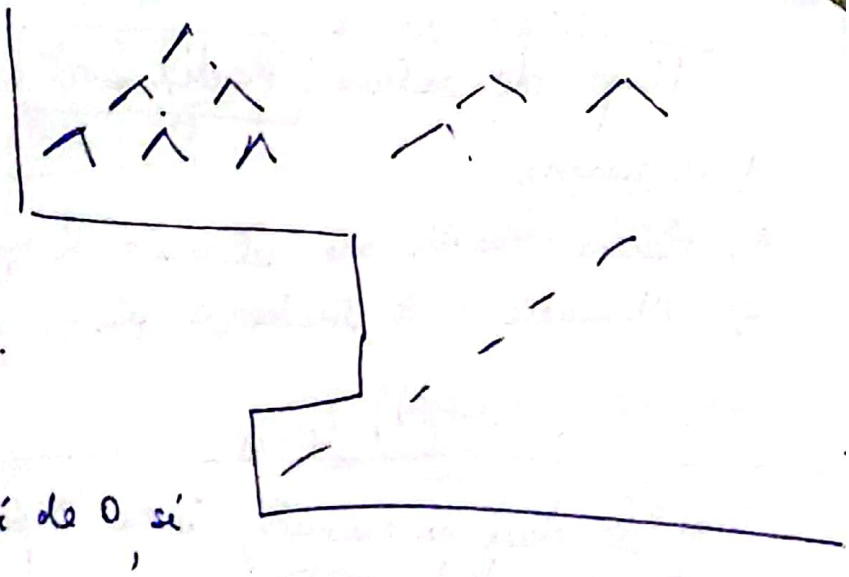
a) $\log_2 n! \in O(n \log n)$

b) $\log_2 n! \in \Omega(n \log n)$

$\log_2 n! \geq \frac{1}{24} \cdot \log_{20} n^n$

Scanned with CamScanner

Count Sort:



Problemă: Avem n nr. de 0 și 1. Cum le putem sorta?

n = 7; 0 1 0 0 0 1 1

Soluție: numărăm câți de 0 și câți de 1 avem.

4 de 0 } => 0 0 0 0 1 1 1
3 de 1 }

Generalizare: Avem n numere din mulțimea $\{0, 1, 2, \dots, K-1\}$. Cum le sortăm?

K = 3
n = 9

0 0 1 2 2 1 0 1 2 ;

0 0 0 1 1 1 2 2 2

vectori de frecvență

0	3
1	3
2	3

$O(n+K)$

Radix Sort

! Un algoritm de sortare stabilă:

Ana 10, Mihaela 10, Ion 9, Mihai 7, Alex 5

Alex 5, Mihai 7, Ion 9, Ana 10, Mihaela 10

Exemplu:

986
123
514
365
777
793
994
420
101
106

Pas I (sortare după cifra unităților)

420
101
123
793
514
994
365
986
106
777

Pas II (sortare după cifra zecilor)

101
106
514
420
123
365
777
986
793
994

Pas III (sortare după cifra sutelor)

101
106
123
365
420
514
777
793
986
994

Scanned with CamScanner

Timp de rulare **Radix Sort**

1) n numere
2) Fiecare număr are cel mult d cifre
3) Numerele sunt în baza k

$O(d(n+k))$

---

1) Se dau n numere între 0 și n^3-1. Cât de repede le putem sorta?

n^3 numere

$n = n$
d cifre $= 3\log_{10} n$
$k = 10$

Radix Sort brut $O(n \log n)$

Transformăm numerele în baza n. !

$\Rightarrow$ $n = n$, $d = 3$, $k = n$

$\Rightarrow$ $O(3(n+n)) = O(n)$

$n > 100$

$0, \ldots, 999999$

Un nr. în baza 100.

$100^1 \cdot 2 + 0 = 20$

Scanned with CamScanner

Alegerea mediaului

```
randomize-select(A, p, r, i)
    if p == r
        return A[p]
    q = randomized-partition(A, p, r)
    k = q - p + 1
    if i == k
        return A[q]
    else if i < k
        return rand_select(A, p, q-1, i)
    else return rand-select(A, q+1, r, i-k)
```

$$T(n) = T(\max(n-k, k-1)) + O(n)$$

$$E(x) = \sum_{i=1}^{n} p_i \cdot v_i$$

$$E(T(n)) \leq E\left(\sum_{k=1}^{n} p_k \cdot T(\max(k-1, n-k)) + O(n)\right)$$

$$= E\left(\frac{1}{n} \sum_{k=1}^{n} T(n-k, k-1)\Big|_{\max} + O(n)\right)$$

(se repetă max)

$$= \frac{1}{n} \cdot 2 \sum_{k=\frac{n}{2}}^{n-1} E(T(k)) + O(n)$$

$$E(T(n)) = O(n) \leq c \cdot n$$

Vrem $E(T(n)) \leq c \cdot n$

Scanned with CamScanner

$$E(T(n)) \le \frac{2}{n} \sum_{k=\lceil \frac{n}{2} \rceil}^{n-1} c \cdot k + a \cdot n$$

$$= \frac{2}{n} \left(\sum_{k=1}^{n-1} ck - \sum_{k=1}^{\lceil \frac{n}{2} \rceil - 1} ck \right) + a \cdot n$$

$$= \frac{2}{n} \cdot c \left(\frac{n(n-1)}{2} - \frac{\lceil \frac{n}{2} \rceil \left(\lceil \frac{n}{2} \rceil - 1 \right)}{2} \right) + a \cdot n$$

$$\cdot \frac{2}{n} \cdot c$$

$$\le cn - \left(\frac{cn}{4} - \frac{c}{2} - an \right)$$

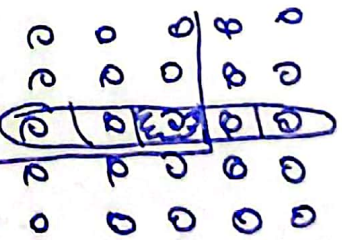
Adv. pt. $\frac{cn}{4} - \frac{c}{2} - an > 0$

Mediana O(n)

V: 2 1 7 5 10 13 → 1 2 5 7 10 13

$|V| = n$

$k = \lceil \frac{n}{2} \rceil$



$\frac{n}{5} \cdot \frac{1}{2} \cdot 3$ el. mai mici ≤ pivot

$\frac{7n}{10}$ el. mai mari pivot

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

$$T(n) \le c \cdot n$$

Scanned with CamScanner

-> Stim că $T(k) \leq ck$, $\forall k < n$

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + n$$

$$\leq c \cdot \frac{n}{5} + c \cdot \frac{7n}{10} + n$$

$$\leq n\left(\frac{c}{5} + \frac{7c}{10} + 1\right) \leq n\left(1 + \frac{9c}{10}\right) \leq cn$$

$1 + \frac{9c}{10} \leq c$

$1 + \frac{9c}{10} - c \leq 0$

$-\frac{c}{10} + 1 \leq 0$

$\forall c \geq 10$

nth-element

$n\left(\frac{1c}{3} + \frac{2c}{3} + 1\right) \leq c$

$c + 1 \leq c$ fals

$T(n) \leq c \cdot n \log n$

$\frac{1}{7} \cdot \frac{1}{2} \cdot n \quad \frac{1}{7} + \frac{5}{7} + 1 \quad c$

$\frac{9c}{7} + 1 \quad \frac{7c}{7} + 1 \leq 0$

$c \leq -\frac{7}{2}$

* 5 4 93 ~~9~~ 23 8
2 1 100 (7) 30 15
↓
k=2

1) Aplicăm alg. de găsire al celui mai mic $k\left(\frac{n}{2}\right)$ ec în timp liniar.
mediana

2) $d_i = (|a_i - Med|)$ și

3) găsim al k-lea cel mai mic el. din d.
4) iterăm prin d și afișăm și eliminăm cei mai mici sau = cu x.

1

X : 1 4 6 8 10 12

Y : 1 2 3 4 5 6

el2 Xmid Ymid el1
sau sau

$x_{mid} = x\left[\frac{1+n}{2}\right]$ între ele căutăm

$y_{mid} = y\left[\frac{1+y}{2}\right]$

a) $x_{mid} = y_{mid} \Rightarrow \frac{x_{mid}+y_{mid}}{2}$

b) $x_{mid} < y_{mid}$

$x_{mid} \ldots a, b \ldots y_{mid}$

$\Rightarrow \begin{cases} x[mid \ldots n] \\ y[1 \ldots mid] \end{cases}$

c) $x_{mid} > y_{mid}$ (Analog invers)

$T(n) = T\left(\frac{n}{2}\right) + 1 = \log n$

mediana a 2 vectori sortați

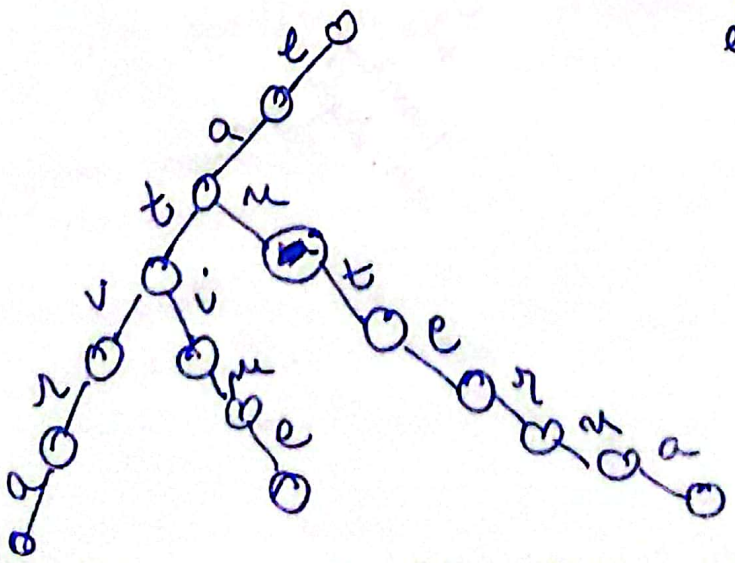
TRIE

latitudine — lungimea lucrării latine

latitudine → prefix comun {2, 3, 4}

latitudine

$O(n \cdot |S|)$ ↓ lung. cuv.



Scanned with CamScanner

Algoritmul de găsire a medianei în O(n) determinist

3 1 7 4 10 2 5
1 2 3 4 5 7 10

2 → 3

La alg. aleator, în cazul favorabil:

$T(n) = T(n/2) + O(n)$
$= O(n)$

În cazul cel mai defavorabil:

$T(n) = T(n-1) + O(n) = O(n^2)$

Algoritmul determinist

2 1 5 10 4 7 5

k = 5
pivot = 5

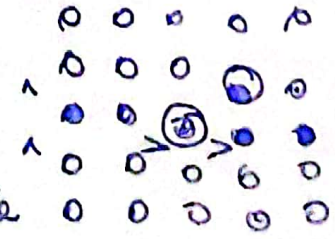
1) Împărțim șirul în grupe de câte 5 (de ce nu 3, 7?)
2) Găsim mediana din fiecare grupă.
3) Găsim mediana celor n/5 mediane (recursiv).
4) Alegem mediana găsită ca pivot și urmăm aceeași strategie ca la alg. recursiv.

4

k = 2
pivot = 4

4 7 5 10

k = 1

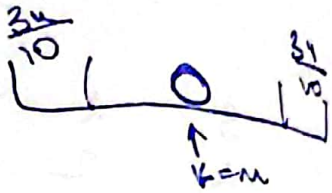


Scanned with CamScanner

$\frac{n}{5}$ grupe cu câte 5 elemente

$\frac{n}{5} \cdot \frac{1}{2}$ grupe care au fiecare câte 3 el. mai mici decât mediana medianelor

↳ $\frac{3n}{10}$ elemente



$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

Vrem $T(n) \leq c \cdot n$

Pp. că $T\left(\frac{n}{5}\right) \leq c \cdot \frac{n}{5}$

$T\left(\frac{7n}{10}\right) \leq c \cdot \frac{7n}{10}$

$$\Rightarrow T(n) \leq c \cdot \frac{n}{5} + c \cdot \frac{7n}{10} + n$$

$$= n\left(\frac{c}{5} + \frac{7c}{10} + 1\right)$$

$$= n \cdot \left(\frac{9c}{10} + 1\right) \leq c \cdot n$$

$\frac{9c}{10} - c + 1 \leq 0$

$\frac{c}{10} \geq 1 \Rightarrow c \geq 10$

$\frac{4}{3} \cdot 2 \quad \frac{n}{3} \rightarrow \frac{2n}{3}$

grupe 3:

$T(n) = T(n/3) + T\left(\frac{2n}{3}\right) + \Theta(n) = O(n \log n)$

grupe 7:

$T(n) = T(n/7) + T\left(\frac{5n}{7}\right) + \Theta(n)$

$\frac{n}{7} \cdot 4 \quad \frac{2n}{7} \quad \frac{5n}{7}$

Temă ⇒

Scanned with CamScanner

Coduri Huffman:

Problema: Se dă un text și vrem să îl codificăm (fiecare caracter să fie reprezentat printr-o secvență de biți) astfel încât numărul total de biți să fie minim.

car.	a	b	c	d	e	f
nr.	45	13	12	16	9	5
*	000	001	010	011	100	101

← cod de lungime fixă.

abaa = 000 001 000 ⇓ 300 biți

*	0	101	100	111	1101	1100

← cod de lungime variabilă

45·1 + 13·3 + 12·3 + 16·3 + 9·4 + 5·4 = 224 biți

```
  32
  32
 ---
  75+
  48
 ---
 123+
  56
 ---
 178+
  45
 ---
 223
```

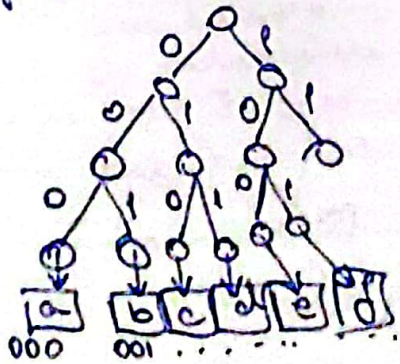
abac = 0 101 0 010

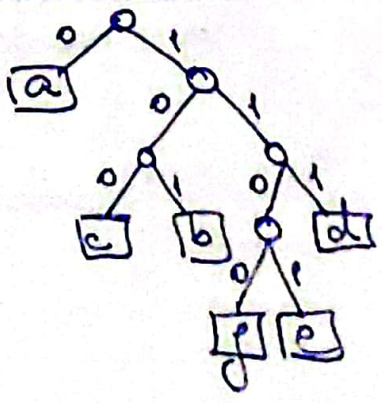
Sc. f = 10 ⇒ 100 → fa / c

→ Pt. a putea decodifica: niciun cod nu trb. să fie prefix al altui cod.

Orice cod poate fi reprezentat ca un arbore binar.

Exemplu: cod fix vs. variabil





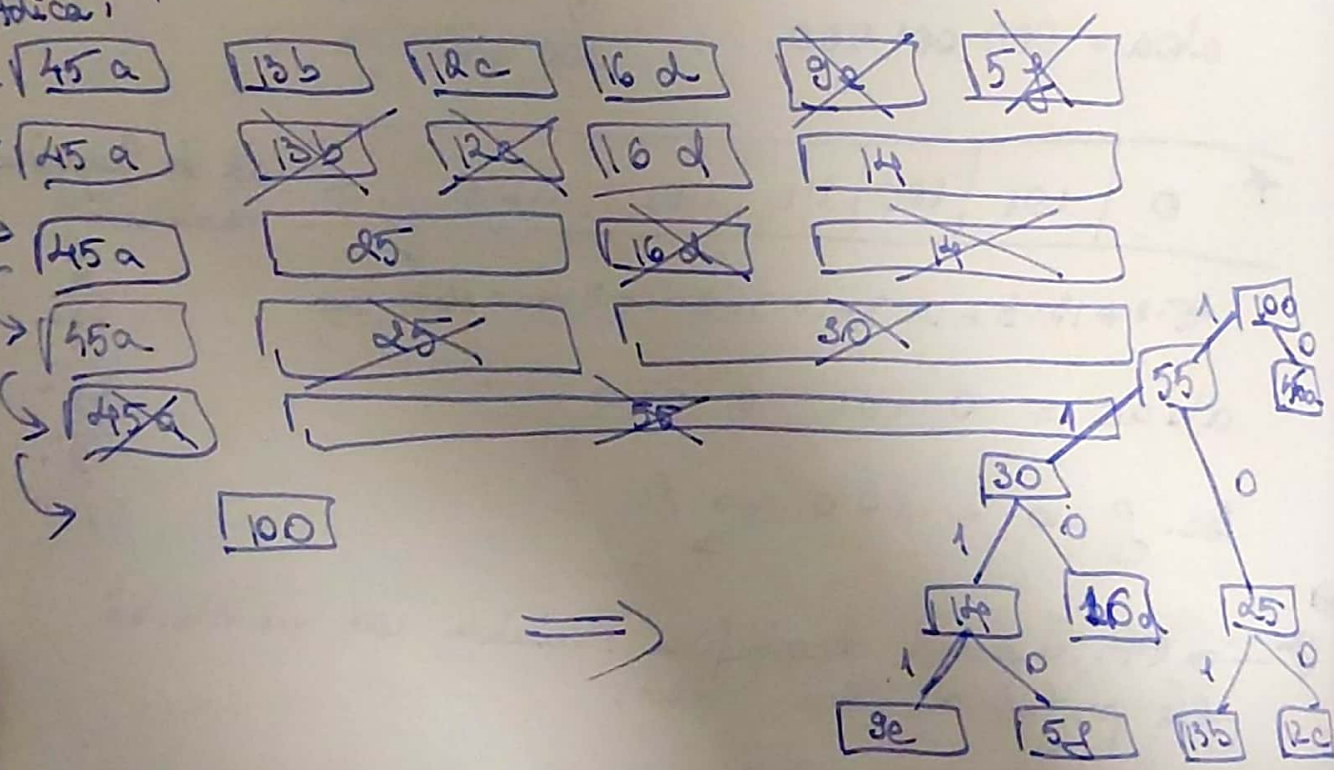
Scanned with CamScanner

Dându-se o mulțime de caractere împreună cu frecvențele asociate, trebuie să găsim codul optim. (lungimea totală a textului codificat să fie minimă).

Algoritm:

1) Toate caracterele vor fi **frunze** în arbore.
2) Alegem 2 noduri cu frecv. **minimă** și adăugăm un nod cu frecvența **însumată** a celor 2 noduri.
→ noul nod = părintele nodurilor alese de frecvență minimă;
3) Repetăm pasul 2 while (nr. noduri ≥ 2)

Adică:



→ Acces la extragerea din **min Heap** și inserții

↓

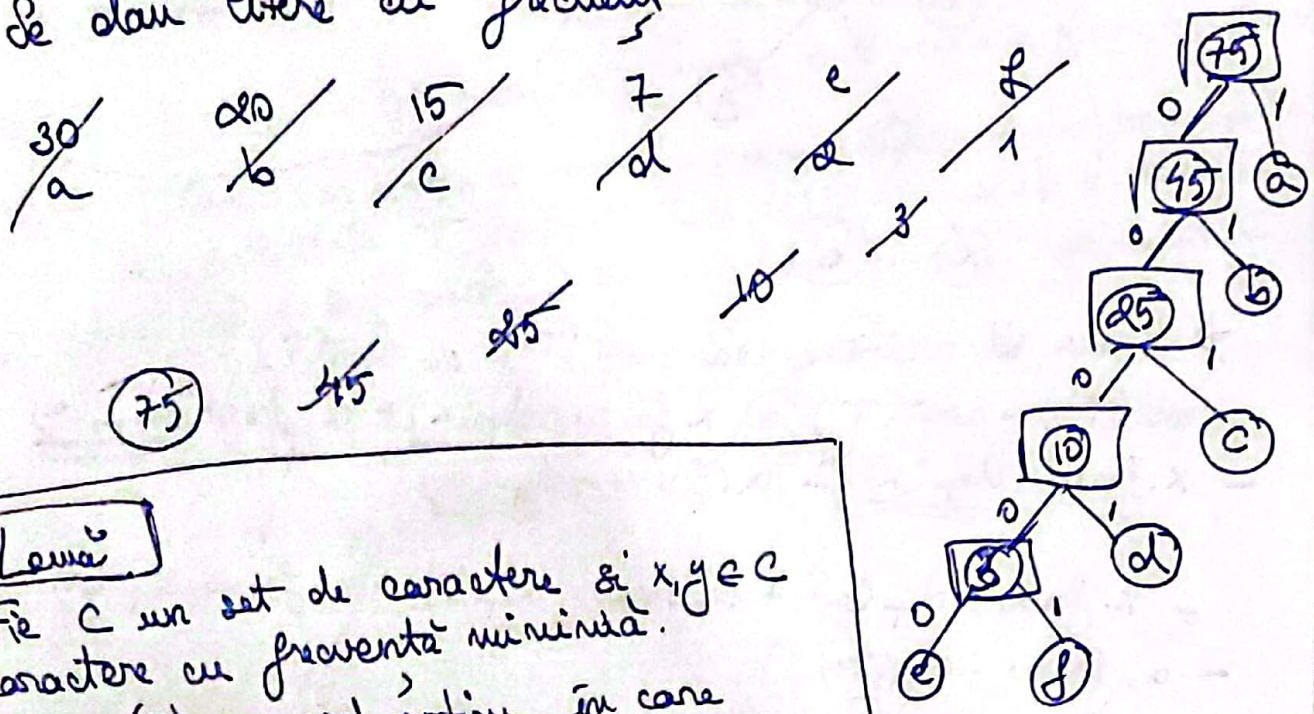
O(n log n)

ab
120

Scanned with CamScanner

Coduri Huffman (4 mai)

Se dau litere cu frecvențe asociate.



Lemă

Fie C un set de caractere și $x, y \in C$ două caractere cu frecvență minimă.

Atunci $(\exists)$ un cod optim în care cuvintele asociate caracterelor x și y au aceeași lungime și diferă doar prin ultimul bit.

Demonstrație (presupunerea abs)

Fie un arbore T corespunzător unui cod optim în care x, y nu sunt frunze de adâncime maximă.

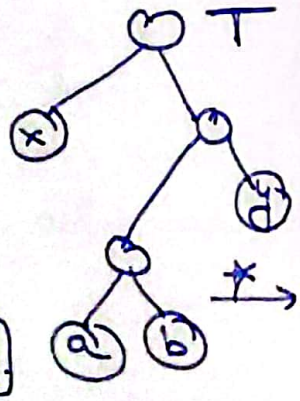
Notații

Fie x, y – caracterele cu frecvență minimă

a, b – caracterele corespunzătoare nodurilor de adâncime maximă

$a.frecv. = frecv.\ caract.\ a$

$d_T(x)$ = adâncimea lui x în arborele T

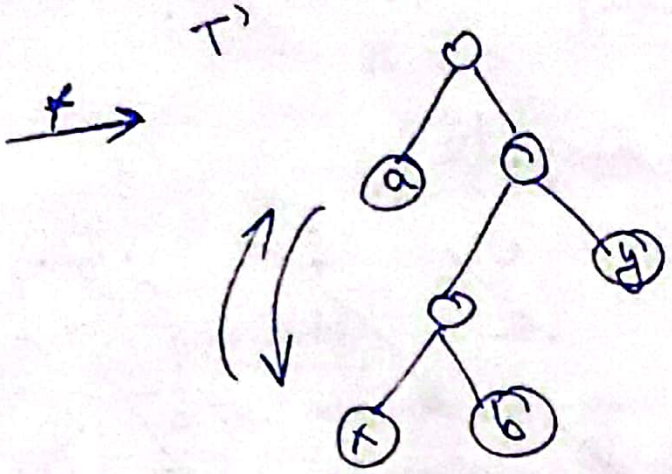


Ideea principală demonstr.

Dacă $(\exists)$ un arbore T în care x și y nu sunt noduri de adâncime maximă, poate construi un arbore T' în care x și y sunt noduri de adâncime maximă și $cost(T') \leq cost(T)$.

unde $cost(T) = \sum_{v \in V} v.frecv \cdot d_T(v)$

Scanned with CamScanner



pune x cu frecv. < mai adânc în arbore
↓
mai eficient

⇒ Vrem să arătăm că $cost(T') \leq cost(T)$.

$cost(T') - cost(T) = x.frecv \cdot \underline{d_{T'}(x)} + a.frecv. \cdot \underline{d_{T'}(a)} - x.frecv \cdot \underline{d_T(x)} - a.frecv. \cdot \underline{d_T(a)}$

$= x.frecv \cdot d_T(a) + a.frecv. \cdot d_T(x) - x.frecv. \cdot d_T(x) - a.frecv \cdot d_T(a)$

$= (x.frecv - a.frecv.)\, d_T(a)$

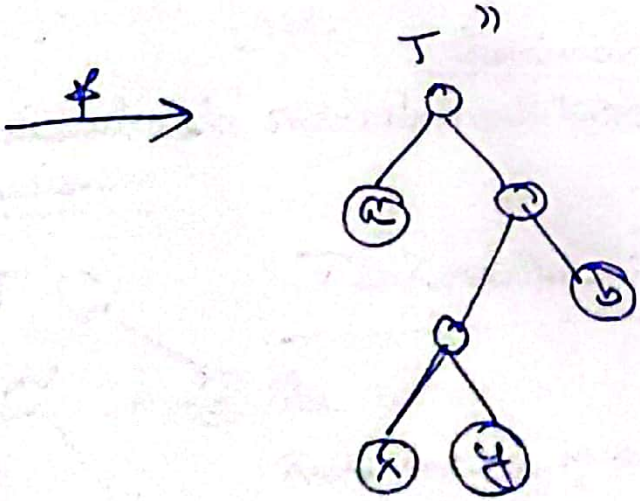
$= x.frecv. (d_T(a) - d_T(x)) + a.frecv. (d_T(x) - d_T(a))$

$= (d_T(a) - d_T(x))(x.frecv. - a.frecv.)$ (?)✓

$d_T(a) - d_T(x) \geq 0$

$x.frecv. - a.frecv. \leq 0$

≤ 0



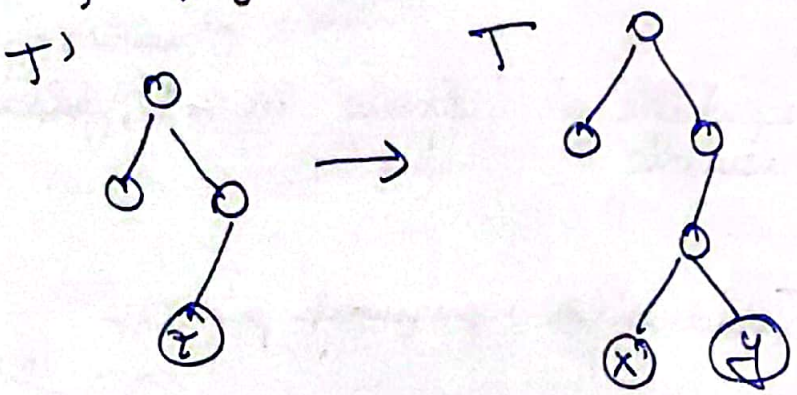
Scanned with CamScanner

Teorema

Fie C un set de caractere și x, y caracterele de frecvență minimă.

Fie $C' = C \setminus \{x, y\} \cup \{z\}$ unde $z.freq = x.freq + y.freq$.

Ac. T' este un arbore ~~optim~~ corespunzător unui cod optim pt. C' at. arborele T format din T' în care z devine nod intern și x și y sunt fii lui z este arbore optim pt. C.



$(cost(T') - cost(T)) = z.freq \cdot d_{T'}(z) - x.freq\,(d_{T'}(z)+1)$
$- y.freq\,(d_{T'}(z)+1) = z.freq \cdot d_{T'}(z) - (d_{T'}(z)+1)(x.freq + y.freq)$

$z.freq$

$= -(x.freq + y.freq)$

$\Rightarrow cost(T) = cost(T') + x.freq + y.freq$.

Pp. red. abs. că ($\exists$) T'' cu $cost(T'') < cost(T)$

Atunci, deoarece T'' este un arbore optim, conform Lemei de la partea I a cursului x, y (caracterele cu freq. minimă vor avea adâncime maximă și vor fi frați)

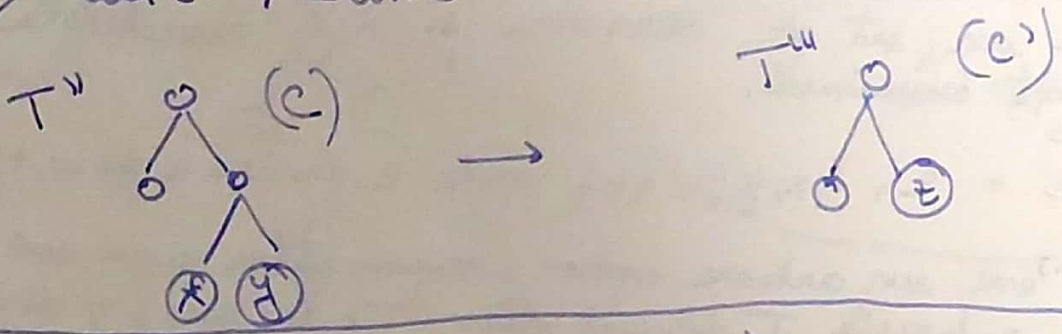
Construim un arbore T''' pentru setul C' care are $cost(T'') = cost(T'') + x.freq + y.freq$

$\Rightarrow cost(T''') = cost(T'') - x.freq - y.freq$

Ac. prin absurd ($\exists$) un arbore optim T'' cu $cost(T'') < cost(T)$
atunci ($\exists$) T''' cu ~~$cost(T''') < cost(T'')$~~
$cost(T''') = cost(T'') - x.freq - y.freq$

Scanned with CamScanner

$\Rightarrow cost(T''') < cost(T')$ ✗



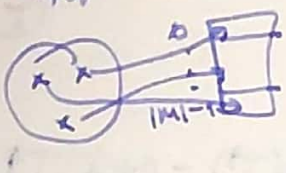
Hashing (tabele de dispersie)

Problema

Vrem să reprezentăm o mulțime $M \subseteq U$, folosind cât mai puțină memorie. (U = univers)

$|M| \ll |U|$

Ex.: mulțimea studenților reprezentați prin CNP



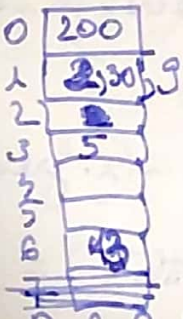
$O(1)$ inserție / căutare

~~Pt. fiecare element x~~

Avem o funcție $h: U \to \{0, \dots, |M|-1\}$

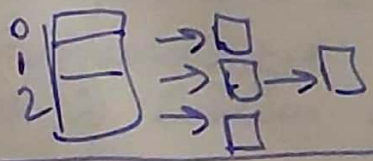
Ex.: $h = (3x+2) \bmod 7$

$M = \{200, 13, 2, 30, 5, 9\}$



Mărimea tabelei este m și cardinalul lui M este n ⇒ cea mai lungă listă are cel puțin $\frac{n}{m}$ elemente.

$n = 7$
$m = 3$



Cum proiectăm o funcție hash bună?

Ce înseamnă o funcție hash?

Dc. mărimea tabelei = m, o funcție hash ar fi de dorit să respecte $P_h[h(x_i) = h(x_j)] = \frac{1}{m}$ pt. orice 2 el. x_i și x_j.

Scanned with CamScanner

În medie lungimea listei i ar fi

$$\sum_{j \neq i} Pr(h_g(x_j) = h(x_i)) = \frac{\text{câte el. am}}{\text{mărimea tabelului}}$$

În loc de o funcție, definesc o familie de funcții cu proprietatea că pt. o funcție $h \in F$, aleasă aleator și $\forall$ elem. x_i și x_j am $Pr(h(x_i) = h(x_j)) \leq \frac{1}{m}$

Ex. de familie de funcții:

$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m^{n-1}$

cu. n = mărimea tabelului

p = nr. prim $> m$

9, 8, 7, 4

~~9~~ ~~8~~ 7 4

~~8~~ ~~9~~ ~~4~~ 7

~~7~~ 8

~~4~~ 9

v[0] = ~~9~~ ~~8~~ 7

index = 1

v[1] = ~~8~~ ~~9~~ 4 1

index = 2

v[2] = ~~7~~ 8

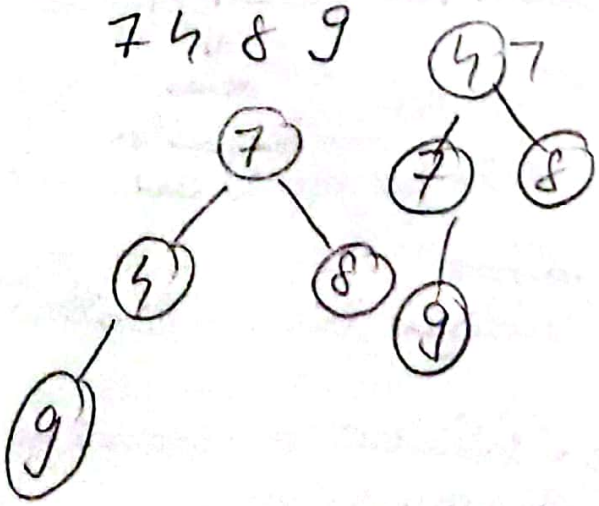
index = 3

v[3] = ~~4~~ 9

v[0] > v[1]

v[0] > [0]

7 4 8 9



Scanned with CamScanner